Kevin Küster

Zusammenfassung: RStr_KL

Rechnerstrukturen

Pipelining & Score-Boards

Pipelining

„Fließband-Prinzip der CPU"; zerlegt Ausführung einer Instruktion in mehrere Phasen, die nacheinander durchlaufen werden

- Pipeline-Phasen einer Instruktion: Fetch Instruction (FI) / Decode Instruction (DI) / Fetch Operand (FO) / Execute Instruction (EI) / Write Operand (WO)

Hazard-Kategorien

- Strukturelle Hazards: Zwei Instruktionen brauchen in derselben Phase dieselbe Hardware-Ressource
- Daten-Hazards: Eine Instruktion hängt vom Ergebnis einer früheren ab
  - ↳ RAW (Read after Write): Echte Datenabhängigkeit; I2 liest register, das I1 schreibt
  - ↳ WAW (Write after Write): Output-Dependency; Beide Schreiben dasselbe Register → Reihenfolge
  - ↳ WAR (Write after Read): Anti-Dependency; I2 überschreibt Register, das I1 liest
- Kontroll-Hazards: Bedingter Sprung verändert Programmzähler
  - ↳ Bereits geholte Instruktionen ggf. obsolet

Scoreboard

Hardware-Buchführungseinheit im Leitwerk, die mehrere Befehle überlappt/parallel ausführen lässt

⇒ Erkennt Datenabhängigkeiten und Ressourcen-Konflikte und verzögert betroffene Befehle

Konflikt-Behebung

- Forwarding (Bypassing): Ergebnis einer Instruktion wird direkt aus dem Latch zwischen Pipeline-Stufen an die folgende Instruktion weitergegeben
  - ↳ Kein Umweg via Register-File
- Register-Renaming: Umbenennung der logischen Register, sd mehrfach verwendete Namen physisch unterschieden werden
  - ↳ löst WAW und WAR-Abhängigkeiten; RAR bleibt bestehen

Beispiel

(1) R4 := R0 * R2
(2) R3 := R1 + R2
(3) R5 := R3 * R4
(4) R3 := R5 + R0

- Pro Zyklus maximal 2 Befehle
- Addition: 2 Zyklen
- Multiplikation: 3 Zyklen
- Schreiben und lesen eines Register im selben Zyklus ✓

RAW-Abhängigkeiten: (3) ← (1) RAW in R4
(3) ← (2) RAW in R3
(4) ← (3) RAW in R5

	Befehl-Nr.		Start Befehl	Ende Befehl	Lesen/Schreiben R0,...,R5	
Zyklus	#	Befehl	Issue	Return	read 012345	Write 012345
1	1	R4 = R0 * R2	1		101000	000010
	2	R3 = R1 + R2	2		211000	000110
2					211000	000110
3					— " —	— " —
4	1	— " —		1	110000	000100
	2			2	000000	000000
5	3	R5 = R4 * R3	3		000110	000001
6					— " —	— " —
7					— " —	— " —
8	3	— " —		3	000000	000000
9	4	R3 = R5 + R0	4		100001	000100
10					— " —	— " —
11	4	— " —		4	000000	000000

Branch Prediction

Technik von Prozessoren, bei der CPU-Prozessor bei bedingtem Sprung vorhersagt, ob dieser ausgeführt wird (taken) oder nicht (not taken) vor Auswertung der Bedingung, damit bereits weitere Befehle einer Pipeline geladen werden können

↳ Vorhersage korrekt: Pipeline läuft weiter; Kein Zeitverlust

↳ Vorhersage nicht korrekt: Pipeline-Flush (kostet Zyklen)

Branch-History-Table (BHT): Hardware-Datenstruktur, die Vorhersage-Bits zu den Branches speichert:

Branch-Adr.	PC-Wert der Branch-Instruktion
Sprungziel (Target)	Adresse des Branch-Ziels; Wird bei taken ohne neue Berechnung verwendet
Status-Bits (History)	1-Bit oder 2-Bit Zustands-Wert

Statisch vs. dynamisch

▷ statische branch prediction: Vorhersage zur Compile-Zeit, feste Regel

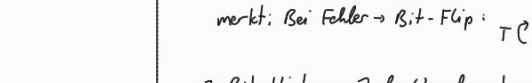
↳ Strategien:

- never taken → Immer raten, dass Sprung nicht genommen wird ⇒ ~30% - 40%
- always taken → Immer raten, dass Sprung genommen wird ⇒ ~60% - 65%

▷ dynamische branch prediction: Vorhersage zur Laufzeit basierend auf vergangener Historie. Branch History Table speichert für jede Branchinstruktion Zustandsautomaten.

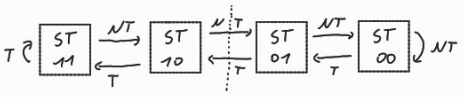
↳ Strategien:

- 1-Bit-History: Pro branch-Instruktion 1 Bit, dass letzten Ausgang merkt; Bei Fehler → Bit-Flip:



- 2-Bit-History: Jede Umschwenkung erhält „zweite Chance" → 2-Bit-Counter → 4 Zustände

ST: Strongly Taken; WT: Weakly Taken; WNT: "NOT"; SNT: "NOT"



Speedup nach Amdahl

Gesetz von Amdahl

Gibt eine obere Schranke Schranke für den Speedup, der durch Parallelisierung erreicht werden kann. → Sequentieller Anteil f als Engpass

⇒ Sequentieller Anteil: Bruchteil eines Programms, der nicht parallelisiert werden kann (Initialisierung, I/O, Synchronisation, kritische Sektoren)

▷ **Amdahl-Formel:**

$$T(n) = \frac{f}{x} + \frac{\alpha}{n \cdot x} \quad ; x = \text{Ausführungsrate (Millions Instructions/s (MIPS))}$$

$$R = \frac{1}{T(n)} = \frac{1}{\frac{n \cdot f}{n \cdot x} + \frac{\alpha}{n \cdot x}} = \frac{n \cdot x}{n \cdot f + \alpha}$$

→ R: Rate = 1/Zeit

→ T(n): Ausführungszeit n Prozessoren (Gesamt)

→ T(1): Ausführungszeit auf 1 Prozessor (=100% normiert) = 1

→ f: Sequentieller Anteil ⇒ $f = \alpha - 1$; $0 \le f \le 1$

→ Paralleler Anteil ⇒ $\alpha = 1 - f$ → Verteilt auf n Prozessoren

▷ **Speedup** (Beschleunigungsfaktor): $S(n) = \frac{T(1)}{T(n)} = \frac{1}{f + \frac{\alpha}{n}}$

→ S(n): Beschleunigungsfaktor bei n Prozessoren

⇒ Randfälle: n = 1 : S(1) = 1 (kein Speedup)

Maximaler Speedup: $\lim_{n \to \infty} : S_{max} = \frac{1}{f}$ (Obere Schranke)

▷ **Effizienz**: Wie gut ist die durchschnittliche Auslastung pro Prozess

$$E(n) = \frac{S(n)}{n} \quad ; E = 1 \hat{=} 100\%, E < 1 : \text{Nicht alle Prozessoren ausgelastet}$$

Beispiel

(i) Wert für α, der zu 40 MIPS führt bei n = 16, x = 4 MIPS

$$R = \frac{1}{T(n)} = \frac{n \cdot x}{n \cdot (1-\alpha) + \alpha} = \frac{x}{1 - \alpha + \frac{\alpha}{n}} \quad (=) \; 40 = \frac{4}{1 - \alpha + \frac{\alpha}{16}}$$

$$(=) \; 1 - \alpha + \frac{\alpha}{16} = \frac{4}{40} \quad (=) \; 1 - \frac{16\alpha}{16} + \frac{\alpha}{16} = \frac{1}{10} \quad (=) \; 1 - \frac{15\alpha}{16} = \frac{1}{10}$$

$$(=) \; \frac{9}{10} = \frac{15\alpha}{16} \quad (=) \; \alpha = \frac{9}{10} \cdot \frac{16}{15} = \frac{144}{150} = 0{,}96$$

(ii) f = 0,2, n für S(n) = 2,5:

$$S_{max} = \frac{1}{\frac{1}{5}} = 5 \rightarrow 2{,}5 \text{ möglich}$$

$$S(n) = \frac{1}{T(n)} = \frac{1}{f + \frac{\alpha}{n}} \quad (=) \; 2{,}5 = \frac{1}{\frac{1}{5} + \frac{\frac{4}{5}}{n}} = \frac{1}{\frac{1}{5} + \frac{4}{5n}}$$

$$= \frac{1}{\frac{n+4}{5n}} = \frac{5n}{n+4} \quad (=) \; 2{,}5n + 10 = 5n \quad (=) \; n = \frac{10}{\frac{25}{10}} = \frac{100}{25} = 4$$

$$\text{Effizienz}: \quad \frac{S(n)}{n} = \frac{2{,}5}{4} = \frac{\frac{25}{10}}{4} = \frac{25}{40} = \frac{5}{8} = 62{,}5\%$$

Superlinearer Speedup

S(n) > n – Gemessener Speedup größer als Anzahl der Prozessoren

↳ Widerspruch Amdahl'sches Gesetz: $S(n) \le n$

▷ Ursachen:

↳ Cache-Effekt: Jeder Prozessor bekommt bei Parallelisierung Teilbereich der Daten → Teilbereich kleiner und passt ggf. vollständig in Cache

↳ Suchraum-Effekt: Bei Parallelen Suche kann Prozessor Lösung in seinen Teilbereichen schneller finden

↳ Zusätzliche Ressourcen: Mehr Prozessoren = Mehr Register/ALUs/...

Verbindungsnetzwerke

Verbindungsnetzwerk ist Verkabelung zwischen Komponenten eines Parallelrechners; es verbindet Prozessoren untereinander mit Speich/Ein-& Ausgabemodulen.

- Statisches Verbindungsnetzwerk: Feste Punkt-zu-Punkt-Verbindung zwischen Knoten (= Prozessoren), modelliert als ungerichteter Graph
- Dynamisches Verbindungsnetzwerk: Verbindungen werden durch Schaltelemente bei Bedarf hergestellt

- Einbettung: Abbildung von Netz NW1 auf Netz NW2 mit eindeutiger Zuordnung der Knoten
- Streckungsgrad: Maximaler Faktor, um den sich Distanzen erhöhen.
  Streckungsgrad = 1 ⇒ nachbarschafts-erhaltend
  ⇒ Zusätzliche Hops (↛ Immer kürzeste Verbindung)

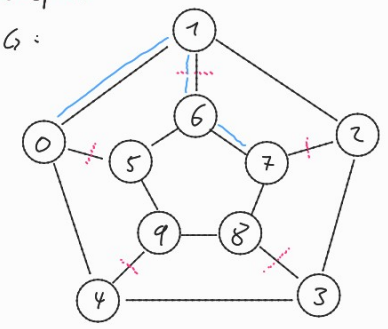
blocking	Manche Permutationen nicht gleichzeitig schaltbar
non-blocking	Jede Permutation gleichzeitig Schaltbar
rearrangeable	Jede Permutation schaltbar, ggf. mit Routing

Bewertungskriterien

- Durchmesser: $\delta(G)$ ist größte kürzeste Distanz zwischen zwei Knoten des Graphen
  - ↳ Distanz = Anzahl der Kanten auf dem kürzesten Pfad
- Grad: $g(G)$ ist die maximale Kantenzahl, die an einem einzigen Knoten hängt
- Bisektionsbandbreite: $B(G)$ = minimale Anzahl an Kanten, die man durchschneiden muss, um den Graphen in zwei gleich große Hälften ($\lfloor n/2 \rfloor$, $\lceil n/2 \rceil$ Knoten) zu teilen
- Knotenkonnektivität: $nc(G)$ = minimale Anzahl an Knoten, die man entfernen muss, damit Graph in getrennte Teile zerfällt
  - ↳ Misst die Ausfallsicherheit ⇒ $nc(G) \leq g(G)$

Beispiel

G:



$\delta(G) = 3$, 0-1-6-7

$g(G) = 3$, Alle Knoten

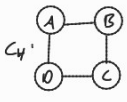
$B(G) = 5$, ----

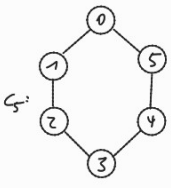
$nc(G) = 3$, ↳ Ein Knoten Isolieren

Einbettungsformen

Ring - C_k:

C_3: 

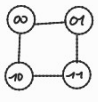
C_4: 

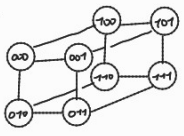
C_5: 

⇒ Grad = 2, $\delta = \lfloor k/2 \rfloor$

Hypercubes - Q_0

Q_1: 

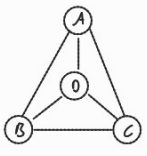
Q_2: 

Q_3: 

⇒ Knoten = 2^d, d-regulär, $\delta = d$

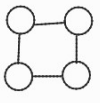
Vollständige Graphen - K_n:

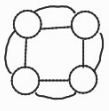
$K_3 = C_3$

K_4 (Tetraeder) 

⇒ $m = n \cdot (n-1)/2$ Kanten,
$g = n-1$, $\delta = 1$

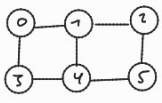
Gitter (Mesh) & Torus:

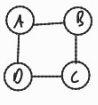
2x2 Mesh 

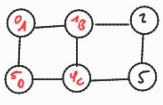
2x2 Torus 

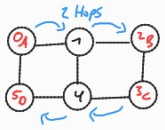
↳ Alle Rand-Knoten haben Wraparound-Kanten

Einbettung:

G: 

H: 

H in G - Streckungsgrad = 1: 

Streckungsgrad = 2: 

Cache-Kohärent

Cache

Kleine, schnelle Zwischenspeicher nahe CPU; Überbrücken Memory Gap - CPU schneller als Hauptspeicher; Caches reduzieren Speicherzugriffszeit durch zeitliche Lokalität (kürzlich verwendetes wird oft wieder gebraucht) und räumliche Lokalität (benachbarte Adressen werden oft zusammen gebraucht)

Kohärenzprotokolle

In Mehrprozessorsystemen hat jeder Prozessor eigenen Cache; Ändert ein Prozessor eine Cache-Line (Kleinste Übertragungseinheit zwischen Hauptspeicher und Cache) lokal, hätte ein anderer Prozessor beim Zugriff veraltete Daten (stale Data) ⇒ Kohärenzprotokolle garantieren Prozessorübergreifend konsistente Sicht auf gemeinsame Daten

⇒ Garantiert, dass alle Prozessoren in einem Shared-Memory-System konsistente Sicht auf gemeinsame Daten haben

- Snoopy: Alle Caches Lauschen (snoop) auf gemeinsamem Bus; Schreibvorgänge per Broadcast. Einfach, skaliert schlecht. → MESI (Modified-Exclusive-Shared-Invalid → 4 Cache-Zustände)
- Directory-Based: Verzeichnis hält Information, welche Caches welche Lines haben; Nur betroffene Caches benachrichtigen. Skaliert besser für NUMA (Non-Uniform Memory Access-verteilter Speicher)

MESI

Snoopy-Kohärenzprotokoll mit 4 Zuständen (2 Bits pro Cache-Line):

Modified (M)	Lokal modifiziert; Nur in diesem Cache; Hauptspeicher veraltet (dirty)
Exclusive (E)	Konsistent mit Hauptspeicher (clean); Nur in diesem Cache
Shared (S)	Konsistent mit Hauptspeicher; Kann in mehreren Caches sein
Invalid (I)	Keine Gültigen Daten

Abwandlung - MOESI: Erweiterung um Owned Flag
↳ Bei Snoop-Read direkter Verweis auf „Besitzer"-Cache ohne Umweg via Hauptspeicher

- Cache-HIT: Eigener Prozessor will Operation auf Cache-Line ausführen (Lesen/Schreiben) → Cache-Line im eigenen Cache vorhanden (→ Gültige Zustände: M, E, S)
  CPU: Zugriff auf Adr. 0x4000 → Cache: Liegt im Zustand E → HIT
- Cache-Miss: -"- → Cache-Line nicht (oder im Zustand I) im eigenen Cache ⇒ Muss zuerst geladen werden
  CPU: -"- → Cache: Kein Zugriff → MISS ⇒ Bus-Anfrage, ggf. Hauptspeicher

Bei MESI-Zustand M: Da eigene Modifikation noch nicht im Hauptspeicher → Zurückschreiben (Copyback) bevor teilen
→ Snoop Write (= RWITM (Read With Intend To Modify) von Außen: Anderer Prozessor will schreiben → Cache

Beispiel

Prozessoren P1, P2 verwenden MESI-Protokoll

Start: P1 = I, P2 = I, Hauptspeicher = x

(i) P2 liest x
- P2 hat I → Read-Miss → Read-Anfrage auf BUS
- P1 snoop → P1 hat I → keine Reaktion
- HS liefert x
- P2 erhält x → kein anderer Cache hält x → P2 = E
- ⇒ P1 = I, P2 = E, HS = x

(ii) P1 schreibt nach x (x → x')
- P1 hat I → Write-Miss → muss x holen und andere invalidieren:
- P1 hat I, will schreiben → sendet RWITM
- P2 snoop RWITM → hat E(x) ⇒ Kein Copyback nötig
  → Invalidiert Kopie: E → I
- P1 lädt x aus HS → schreibt x' (lokal) → I → M(x')
- HS: nicht aktuell
- ⇒ P1 = M(x'), P2 = I, HS = x (veraltet)

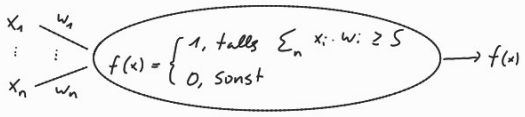
(iii) P1 schreibt nach x (x → x")
- P1 hat M → Write-Hit → kein Bus-Verkehr nötig
- ↳ P1 alleiniger Inhaber; schreibt lokal x' → x"
- P1 bleibt in M
- ⇒ P1 = M(x"), P2 = I, HS = x (veraltet)

(iv) P2 liest x
- P2 hat I → Read-Miss → P1 hat aktuelle Version im Dirty State:
- P2 hat I → Read-Miss → Read-Anfrage auf BUS
- P1 snoop → hat M(x") → HITM (Hit Modified)
  - → HS Zugriff wird unterbrochen
  - → P1 schreibt x" auf HS (Dirty line Copyback)
  - → P1 wechselt M → S
- HS: aktualisiert x"
- P2: Erhält x" von BUS → P2 = S
- ⇒ P1 = S(x"), P2 = S(x") HS = x"

Künstlich Neuronale Netze

MP-Neuron

McCulloch-Pitts-Neuron (MP-Neuron): Erstes mathematisches Modell eines Neurons; Binäres Schaltelement mit festem Schwellwert

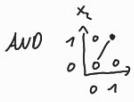
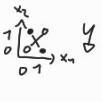


$x_1, \dots, x_n$: Eingaben — S: Schwellenwert

$w_1, \dots, w_n$: Gewichtungen der Eingaben — $f(x)$: Ausgabefunktion

SLP & MLP

- Single Layer Perceptron (SLP): Ein-Schichtiges Netz:
  Eingabe → Ausgabe (-Neuron(en)); Kann nur linear trennbare Funktionen abbilden ⇒ AND, OR ✓ ; XOR ✗
  ⇒ Training via Perceptron-Algorithmus:
  Bei falscher Ausgabe werden Gewichte angepasst:
  Fehler $0 \rightarrow 1$: $w_i = w_i + \eta \cdot x_i$
  Fehler $1 \rightarrow 0$: $w_i = w_i - \eta \cdot x_i$
  → Forward-Pass: Ein Eingabe-Vektor wird schichtweise durch das Netz weitergeleitet bis zur Ausgabe
- Multi Layer Perceptron (MLP): Mehrschichtiges Netz mit mindestens einem Hidden Layer zwischen Ein- & Ausgabe. Kann beliebige Funktionen approximieren, sofern genug Neuronen
  ⇒ Training via Backpropagation-Algorithmus:
  Fehler an Ausgabe wird rückwärts durchs Netz propagiert;
  Jedes Gewicht erhält Gradienten der angibt, wie sehr es zum Fehler beigetragen hat; Gewichte entsprechend korrigieren
  → Wird mathematisch über die Kettenregel realisiert via Verlustfunktion
  ⇒ Setzt differenzierbare Funktion voraus
  ⇒ Optimierungsziel: Minimierung der Verlustfunktion
- Linear Trennbar: Boolesche Funktion $f(x_1, x_2)$ linear trennbar, wenn es Gerade gibt, die Punkte $f=1$ und Punkte $f=0$ trennt

AND  OR  XOR 

= Linearer Klassifaktor als Hyperebene zur Ergebnis-Trennung

Darstellung Logischer Operatoren

Eingaben: x_1, x_2

AND: $y=1$ bei $(1,1)$
↳ $w_1 = w_2 = 1$, $S = 1,5$
$(0,0) = 0 < 1,5 \rightarrow 0$
$(0,1) = 1 < 1,5 \rightarrow 0$
$(1,0) = 1 < 1,5 \rightarrow 0$
$(1,1) = 2 \geq 1,5 \rightarrow 1$

OR: $y=1$ bei $(0,1), (1,0), (1,1)$
↳ $w_1 = w_2 = 1$, $S = 0,5$

NOR: $y=1$ bei $(0,0)$
↳ $w_1 = w_2 = -1$, $S = -0,5$

XOR:
Hidden Layer:
$h_1 = AND(x_1, NOT(x_2))$
$h_2 = AND(NOT(x_1), x_2)$
Outer Layer:
$y = OR(h_1, h_2)$

Lernmethoden

- Überwachtes Lernen (Supervised learning): Trainingsdaten enthalten Eingaben und zugehörige Soll-Ausgaben (=Labels). Netz lernt Zuordnung Eingabe → Label;
  Anwendung: Bildklassifikation, Spracherkennung
  Algorithmen: Perceptron-Lernregel, Backpropagation
- Unüberwachtes Lernen (Unsupervised learning): Nur Eingaben, keine Labels;
  ↳ Das Netz entdeckt selbst Strukturen
  Anwendung: Cluster-Bildung, Dimensionsreduktion
  Algorithmen: Self-Organizing Maps, Adaptive Resonance Theory
- Bestärkendes Lernen (Reinforcement learning): Feedback durch Belohnung/Bestrafung; Netz lernt Strategie (Policy) zur Maximierung der Gesamt-Belohnung
  Anwendung: Spiel-Agenten (Schach,...), Roboter-Steuerung
  Algorithmen: Q-Learning, Deep-Q-Network

Multithreading & Superskalarität

Superskalar

Prozessor, der pro Takt mehr als eine Instruktion ausführen kann; Dazu sind mehrere Funktionseinheiten parallel aktiv (z.B. mehrere ALUs)

=> Instruction per Cycle (IPC) > 1 möglich

Very long Instruction Word (VLIW)

Sehr lange Befehlsworte, die mehrere Operationen kodieren; Compiler entscheidet zur Übersetzungszeit, welche Operationen parallel ausgeführt werden sollen;

↳ pro: einfache Hardware (kein Out-Of-Order, kein Reorder Buffer); deterministisches Timing

↳ contra: Compiler trägt die Komplexität; Code-Größe wächst (NoOperation-Instruktion für unbelegte Slots); Abhängig von Speicher-Latenz, die Compiler nicht mehr kennt

Multithreading - Varianten

- Fine-Grained Multithreading (FMT): In jedem Takt wird ein anderer Thread ausgeführt; Latenz-Stalls eines Threads werden durch andere Threads überbrückt
- Coarse-Grained Multithreading (CMT) / Block-Multithreading: Ein Thread läuft so lange, bis er auf eine Latenz wartet (z.B. Cache-Miss); Dann wechselt Prozessor auf anderen Thread
- Simultaneous Multi-Threading (SMT): Im selben Takt werden Instruktionen von mehreren Threads gleichzeitig ausgegeben; Funktioniert auf superskalaren Prozessoren mit ungenutzten Slots.

CISC/RISC, SPRS

Complex Instruction Set Computer (CISC)

Architektur mit großem, komplexem Befehlssatz; Variable Befehlslänge, viele Adressierungsarten, mikroprogrammierte Steuerung

Reduced Instruction Set Computer (RISC)

Architektur mit kleinem, einfachem Befehlssatz; Einheitliche Befehlslänge, Load/Store-Architektur, wenige Adressierungsarten, festverdrahtete Steuerung

↳ Befehle fester/gleicher Länge + einheitlicher (einfacher) Aufbau => gute Voraussetzungen für Pipelining

↳ Gleichzeitig höher Verlezung in Maschinenbefehle aufgrund mangelnder Komplexität

CPU-Time

CPU time = IC · CPI · T_C

↳ IC: Instruction Count (RISC: höher; CISC: niedriger)

↳ CPI: Cycles per Instruction (RISC: ≈1; CISC: >5) → Dadurch RISC schneller

↳ T_C: Clock Cycle Time (RISC niedriger durch verdrahtete Steuerung)